Principles of Machine Learning

Mariette Schönfeld (session 1,3,5) *mariette.schoenfeld@kuleuven.be*
Andrea Mirenda session (2,4) *andrea.mirenda@kuleuven.be*
Isidoro Tamassia (session 6,7) *isidoro.tamassia@kuleuven.be*

Fall, 2026

Exercise Session 5: Artificial Neural Networks

5.1 Representation Power

Assume the inputs and outputs of a neural network are boolean values, represented by the numbers -1 and 1. If the activation function is the sign function

$$\sigma(x) = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$$

then give a 2-input perceptron that represents the following functions, if possible.

1. $(A, B) \mapsto A \vee B$
2. $(A, B) \mapsto A \Leftrightarrow B$
3. $(A, B) \mapsto \neg A \wedge B$

Now consider a boolean function $f : \mathbb{B}^n \to \mathbb{B}$ in conjunctive normal form with k clauses

$$f(A_1, ..., A_n) = \bigwedge_{i=1}^{k} \bigvee_{j=1}^{n} f_{ij}(A_j)$$

where $f_{ij}(A) \in \{A, \neg A, \text{false}\}$ for all i and j. Give a neural network with two layers and n inputs that represents this function.

Solution 5.1 A 2-input perceptron p represents any function of the form

$$(x, y) \mapsto \sigma(ax + by + c)$$

where a, b and c are the weights that can be varied. By choosing $a = 1$, $b = 1$ and $c = 3/2$, we can represent the disjunction of two inputs. If we however set $a = -1$, $b = 1$ and $c = -3/2$, then we obtain the third function. It is not possible to represent the equivalence function, since a perceptron, essentially a linear separator, cannot separate four points on a square of which the opposite points have the same labellings, but different from the other pair of opposites.

For a general boolean function in conjunctive normal form, consider the following architecture. We add one neuron ν_i to the hidden layer for every clause i in the conjunction. If a variable A_j occurs positively in clause i, meaning $f_{ij}(A) = A$, we set the weight $w[A_j \to \nu_i] = 1$. If it occurs negatively, $f_{ij}(A) = \neg A$, we choose $w[A_j \to \nu_i] = -1$ and if it does not occur $w[A_j \to \nu_i] = 0$. The offset for ν_i is set to $n_i - 1/2$ where n_i is the number of variables occuring in clause i. The node ν_i will thus be activated if the clause is true. The weights for the output node ν are all set to $w[\nu_i \to \nu] = 1$ and its offset is $-k + 1/2$ with k the total number of clauses. Note that the $1/2$ terms in the offsets are merely added to avoid the edge cases of evaluating $\sigma(0)$ and are not fundamental. ◇

5.2 Decision Surfaces

Consider the decision surfaces in figure 1 below. For which of these surfaces can we find a 2-layer neural network that exactly represents this decision surface? The network takes 2-dimensional real input and uses the sign function as activation function in the hidden layer. Can you represent any convex polygon? Assume you know the equations of the individual lines.

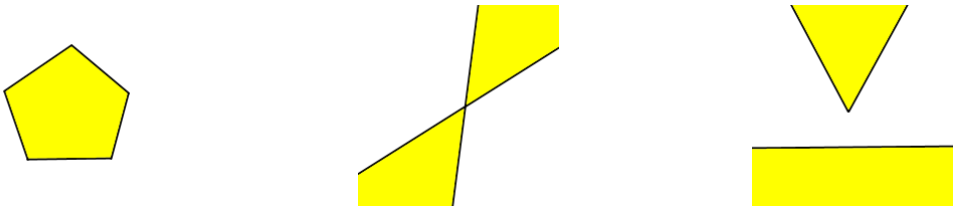


Figure 1: Three decision surfaces for 2-dimensional input.

Solution 5.2 In the first layer of the neural network, we can add neurons that represent on which side of a line a point lies. If we have a line L given by the equation $ax + by + c = 0$, we set the weights and offset to $w[x \to \nu_L] = a$, $w[y \to \nu_L] = b$ and $q[\nu_L] = c$ with ν_L the neuron in the hidden layer corresponding to line L. A point lies within a polygon only if it lies on the correct side of each edge of that polygon, namely the side towards the centre of the polygon. By combining the neurons in the first layer in a conjunction we can exactly model this condition. To do this we set the weights $w[\nu_L \to \nu] = 1$ and $q[\nu] = -l + 1/2$ with l the number of edges in the polygon and ν the output neuron.

The second decision surface cannot be represented exactly. To do so we would require the second layer to output the equivalence function, and as we have seen in the previous exercise, this is not possible with a single layer. According to the universal approximation theorem however, it is possible to approximate the surface to arbitrary precision on any compact set.

The third surface can be represented in the first layer by again choosing the weights corresponding to the three lines. Assume the node α fires when the point lies below the bottom line, the node β fires when it lies right of the top left line and γ fires when it lies left of the top right line. In the second layer we then set the weights $w[\alpha \to \nu] = 2$, $w[\beta \to \nu] = 1$, $w[\gamma \to \nu] = 1$ and $q[\nu] = 3/2$. By doing this, the output node ν will fire when $\alpha = 1$ or if $\beta = 1 = \gamma$, covering exactly the desired decision surface. ◇

5.3 Gradient Backpropagation

Consider the simple network in Figure 2 which has a single real input X, a hidden node H and an output Y. Let us use a sigmoid $s \mapsto (1 + e^{-s})^{-1}$ as activation function in the hidden layer and a linear function in the output layer (in other words: in the hidden layer, an activation is applied to a linear function. It is the same for the output, without the activation). Write down H as a function of X and Y as a function of H. Using gradient backpropagation, calculate the gradient of Y with respect to every parameter in the network.

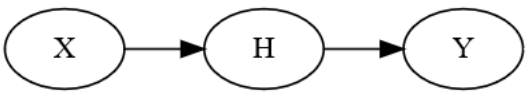


Figure 2: Simple neural network to illustrate backpropagation.

Solution 5.3 First we write

$$H = \frac{1}{1 + \exp(-w_H X - q_H)} \quad \text{and} \quad Y = w_Y H + q_Y$$

with w_H, q_H, w_Y and q_Y the parameters of the network. The gradients with respect to the last two parameters can be immediately obtained as

$$\frac{\partial Y}{\partial w_Y} = H = \frac{1}{1+\exp(-w_H X - q_H)} \quad \text{and} \quad \frac{\partial Y}{\partial q_Y} = 1$$

whereas for the first two parameters we first need to calculate $\partial Y/\partial H = w_Y$ which in turn is used to find the expression

$$\frac{\partial Y}{\partial w_H} = \frac{\partial Y}{\partial H}\frac{\partial H}{\partial w_H} = w_y \frac{X\exp(-w_H X - q_H)}{(1+\exp(-w_H X - q_H))^2} \quad \text{and} \quad \frac{\partial Y}{\partial q_H} = \frac{\partial Y}{\partial H}\frac{\partial H}{\partial q_H} = w_y \frac{\exp(-w_H X - q_H)}{(1+\exp(-w_H X - q_H))^2}$$

for the first two gradients. We can see that the gradient backpropagates from Y through H to w_H and q_H. ◇

5.4 Stochastic Gradient Descent

The neural network in Figure 3 has a rectified linear unit ρ as activation function in each layer, which is defined as:

$$\rho(x) = \begin{cases} x & x > 0, \\ 0 & x \leq 0. \end{cases}$$

We will ignore the non-differentiable point at zero. The current values for the weights w are indicated on the edges and the offset terms q given in the nodes. We will perform stochastic gradient descent to learn the weights. The difference with regular gradient descent is that stochastic gradient descent only uses a randomly chosen subset of the data to calculate the gradient. In the next iteration, a new subset is again chosen at random. In our case, we will use only a single data point for one iteration. For input $(x_1, x_2) = (5, 3)$ we know the target value of $\hat{y} = 1$. Assume we have sampled this data point for stochastic gradient descent. Using the squared loss function $(y(w, q) - \hat{y})^2$, perform one update step on the weights w and offsets q, also called the bias term. Assume a learning rate η of unity.

Consider now the case that some weights are shared. If we require w_2 and w_3 to be the same and initialise this common weight to $w_2 = -1 = w_3$, what is the update to this weight?

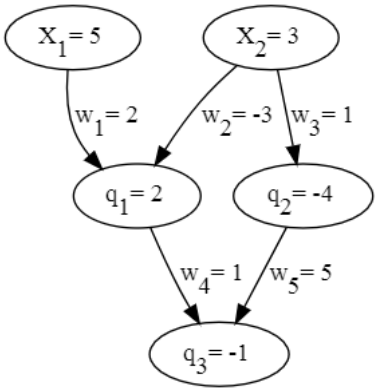


Figure 3: Neural network on which we can perform gradient descent.

Solution 5.4 We first calculate the value of the hidden nodes $h_1 = 3$, $h_2 = 0$ and the output node $y = 2$ using the given weights and offsets. By using the chain rule, we can now calculate all the derivatives. For example,

$$\frac{\partial y}{\partial w_1} = \frac{\partial y}{\partial h_1}\frac{\partial h_1}{\partial w_1} = w_4 \frac{\partial}{\partial w_1}\rho(w_1 x_1 + w_2 x_2 + q_1) = w_4 x_1 = 5$$

and

$$\frac{\partial y}{\partial q_2} = \frac{\partial y}{\partial h_2}\frac{\partial h_2}{\partial q_2} = w_5 \frac{\partial}{\partial q_2}\rho(w_3 x_2 + q_2) = w_5 \cdot 0 = 0$$

where the derivative vanishes for q_2 because $\rho(w_3 x_2 + q_2)$ is zero for the given values of w_3, x_2 and q_2. Similarly the other derivatives can be calculated and are

$$\frac{\partial y}{\partial w_1} = 5 \qquad \frac{\partial y}{\partial w_2} = 3 \qquad \frac{\partial y}{\partial w_3} = 0$$
$$\frac{\partial y}{\partial w_4} = 3 \qquad \frac{\partial y}{\partial w_5} = 0$$
$$\frac{\partial y}{\partial q_1} = 1 \qquad \frac{\partial y}{\partial q_2} = 0 \qquad \frac{\partial y}{\partial q_3} = 1$$

which we combine into the gradient. To minimize the loss, we have to update the parameters according to

$$w' = w - 2\eta \frac{\partial y}{\partial w}(y - \hat{y})$$

in which $(y - \hat{y}) = 1$ for our example. This leads to new parameters

$$w_1' = -8 \qquad w_2' = -9 \qquad w_3' = 1$$
$$w_4' = -5 \qquad w_5' = 5$$
$$q_1' = 0 \qquad q_2' = -4 \qquad q_3' = -3$$

which have changed quite a bit due to the large learning rate. In practice, one should choose a smaller learning rate.

If $w_2 = -1 = w_3$ we have to consider both branches to calculate the gradient

$$\frac{\partial y}{\partial w_2} = \frac{\partial y}{\partial h_1}\frac{\partial h_1}{\partial w_2} + \frac{\partial y}{\partial h_2}\frac{\partial h_2}{\partial w_2} = w_4 x_2 + w_5 \cdot 0 = 3$$

which results in an update

$$w_2' = w_2 - 2\eta \frac{\partial y}{\partial w_2}(y - \hat{y}) = -1 - 2 \cdot 1 \cdot 3 \cdot (8 - 1) = -43$$

where it should also be noted that the values for y and h_1 have changed due to the change in the weight. Once again this update step should be damped by setting a smaller η in practice. ◇

5.5 Universal Approximation

Let us prove that neural networks are universal approximators for arbitrary continuous functions $f : [0, 1] \to \mathbb{R}$. For this purpose we make use of the known result that continuous functions on bounded closed intervals can be uniformly approximated to arbitrary precision by step functions of the form

$$S(x) = \sum_{n=1}^{N} S_n \chi[x \in [a_n, b_n)]$$

with $a_n < b_n$ for all n. In this notation χ is an indicator function, so $\chi[x \in [a, b)] = 1$ if $x \in [a, b)$ and zero otherwise. Convince yourself of this result. You do not need to prove it formally.

It thus sufficient to show that a neural network with a single in- and output and one hidden layer can approximate step functions to arbitrary precision. Consider neural networks with two layers, an activation function

$$\sigma(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

in the hidden layer and a linear activation function in the output layer. Show that a suitable choice of the weights and offsets allows us to exactly represent any step function in the above form.

In fact, this proves the one-dimensional case of the universal approximation theorem. The general theorem states that any continuous function $f : \mathbb{R}^d \to \mathbb{R}$ in any dimension d can be approximated

uniformly to arbitrary precision on compact sets by a neural network with only one hidden layer. The general proof is a bit more involved however. Furthermore notice that decision trees also have the power to represent arbitrary step functions and, by the same arguments, are universal approximators too.

Solution 5.5 Take an arbitrary step function S in the above form with N steps. For every step n we will add a pair of neurons ν_n and μ_n to the hidden layer for a total of $2N$ neurons. The first neuron in the pair of a given step has a weight $w[X \to \nu_n] = 1$ and offset $q[\nu_n] = -a_n$. In other words, it fires whenever $x \geq a_n$. The second neuron in the pair also has weight $w[X \to \mu_n] = 1$ but a different offset $q[\mu_n] = -b_n$. The weights to the output neuron are given by $w[\nu_n \to \nu] = S_n$ and $w[\mu_n \to \nu] = -S_n$. The offset is zero for the output neuron. One can check that this network represents exactly the step function S. ◇

5.6 Convolutional Neural Networks

We want to classify d by d images with k channels using a convolutional neural network. Our architecture consists of a convolutional layer λ_1 with n filters, each having kernel size $2s + 1$. The kernel size is the dimension of the square filter that is used in the convolution. Following this layer is a max pooling layer λ_2 with pool size l. If the input to our network is x, what are the dimensions of x, $\lambda_1(x)$ and $\lambda_2(\lambda_1(x))$? Assume that no padding is performed.

How many weights are present in this neural network? How many weights would there be if we used a fully connected layer instead of a convolutional layer, assuming the target dimensions remain the same? For a typical setting where $d = 64$, $k = 3$, $n = 5$, $s = 1$ and $l = 2$, what is the fractional reduction in the number of weights by choosing a convolutional layer over a fully connected layer?

Solution 5.6 We start with input x having dimensions $d \times d \times k$. After applying one convolutional filter to this image, we get an image with dimensions $(d - 2s) \times (d - 2s)$. Doing this for all n filters yields a dimension of $(d - 2s) \times (d - 2s) \times n$ for $\lambda_1(x)$. A max pooling layer combines regions of l by l values into one value, so we get a contraction by a factor of l in each dimension. As a result $\lambda_2(\lambda_1(x))$ will have dimensions $(d - 2s)/l \times (d - 2s)/l \times n$.

The convolutional network will have $k(2s + 1)^2$ weights for each of the n filters, giving a total of $nk(2s+1)^2$ weights. For a fully connected layer there would be a connection weight running from each of the kd^2 input nodes to the $n(d - 2s)^2$ nodes in the first layer, adding up to $nkd^2(d - 2s)^2$ weights. The fractional decrease is thus

$$\frac{nkd^2(d-2s)^2 - nk(2s+1)^2}{nkd^2(d-2s)^2} = 1 - \frac{(2s+1)^2}{d^2(d-2s)^2} = 0.99999943...$$

which is a drastic reduction in the number of weights. ◇

5.7 Auto-Encoders

Consider an auto-encoder that encodes n-dimensional input into an encoded layer with $m < n$ nodes. If all the activation functions are linear, argue that training such an auto-encoder is equivalent to performing a principal component analysis on the data. How many components do you keep?

Solution 5.7 The auto-encoder consists of a linear encoding map $E \in \mathbb{R}^{m \times n}$ and a decoding map $D \in \mathbb{R}^{n \times m}$. The auto-encoder thus maps its input first to an m-dimensional space and then back to n dimensions. Since $m < n$ not all components of the input can be kept, but only m components, depending on the matrices E and D. It is thus best to keep only the m most prominent components in the data and the optimization algorithm should let E converge to a matrix containing the m most prominent directions in its rows. This is similar to what a principal component analysis does. ◇

5.8 Hopfield Networks

Hopfield networks are special types of neural networks that try to mimic a memory. During training, patterns can be given, which will determine the connection weights of the network. When given a new input, the network finds the closest pattern to that input by running iterative updates on the input until it converges to some fixed state, hopefully one of the training patterns.

For example, if we want to recognize images representing digits, we can train a Hopfield network with one archetypical example for each digit. These will be stored as patterns. When given a new image, the network will evolve this input image until it converges to one of the archetype digits. This archetype can be used to classify the new image.

Like LSTM networks, Hopfield networks try to mimic memory. However, due to the update rule of Hopfield networks, so called spurious patterns can occur. These are unwanted extra patterns that are stored in the network. This is one of the reasons LSTM network have become more popular compared to Hopfield networks. Let us now look at how the evolution of a Hopfield network takes place.

For d-dimensional input, the constructed Hopfield network will have d fully connected neurons, each neuron representing one component of the input. The value of these neurons x will be updated from time n to time $n+1$ according to the update rule

$$x^{(n+1)} = \alpha(Wx^{(n)} + q)$$

where α is the activation function, W is the weight matrix and q is the offset. For α one often chooses a saturated linear function

$$\alpha(x) = \begin{cases} -1 & x < -1 \\ 1 & x > 1 \\ x & \text{otherwise} \end{cases}$$

and the weights and offset are learned during the training phase. One way to set the weights is with the Hebbian learning rule, which states that for a set of patterns S to be learned, define the weights

$$W_{ij} = \sum_{s \in S} s_i s_j$$

and set the offset to zero.

Given a 2-dimensional input space and the patterns $S = \{(1,1),(1,-1),(-1,-1)\}$, apply the Hebbian learning rule to obtain the weight matrix W. Are the patterns fixed points of the update rule?

Now consider the input $(-1/2, 2/3)$ and perform three update steps on it. To what value does it converge? Can you explain this strange behaviour?

Solution 5.8 A direct application of the formula for the weights results in

$$W = \sum_{s \in S} ss^{\mathrm{T}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & -1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \end{bmatrix} \begin{bmatrix} -1 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

which we can apply to check whether the patterns are fixed points. This is indeed the case, since

$$\alpha\left(\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$
$$\alpha\left(\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$
$$\alpha\left(\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

where α applied to a vector means applying it to each component separately.

Performing three update steps to the input $x^{(0)} = (-1/2, 2/3)$ yields

$$x^{(0)} = \begin{bmatrix} -1/2 \\ 2/3 \end{bmatrix} \quad x^{(1)} = \begin{bmatrix} -5/6 \\ 1 \end{bmatrix} \quad x^{(2)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad x^{(3)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

revealing that it converged to $(-1, 1)$, which turns out to be a fixed point as well. To explain this, notice that the evolution of an input is symmetric under negation. Since α is an odd function and the offset is set to zero, the update rule has the property that a negated input leads to a negated output,

$$\alpha(W(-x^{(n)})) = \alpha(-Wx^{(n)}) = -\alpha(Wx^{(n)}) = -x^{(n+1)}$$

and thus if some point x is fixed under the update rule, then $-x$ will be fixed as well. Since $(1, -1)$ is fixed, we conclude that $(-1, 1)$ will be so too, as we have observed. ◇

5.9 Keras

The file neural.py can be found online. Implement the missing method, used to define and train a convolutional neural network. You can then change the parameters and see what effect each has on the training process.